Department of Computer Science and Engineering
Assignment Questions

Course Code & Name : DSA01 - Object Oriented Programming with C++

Assignment Title:	Urban Complaint Management and Service Request System
Course Outcome(s) – CO:	**CO3:** Construct C++ programs using constructors, destructors, and operator overloading mechanisms to control object creation, destruction, and operator behaviour . **CO4:** Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers, and polymorphism to design reusable and extensible programs.
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse, L5-Evaluate
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 16 – Peace, Justice and Strong Institutions; SDG 12 – Responsible Consumption and Production

Problem Statement: Design, implement, and evaluate a menu-driven, object-oriented C++ application for an Urban Complaint Management and Service Request System to automate the registration, categorization, prioritization, assignment, status tracking, and resolution of civic complaints such as road damage, water supply, waste management, drainage, and streetlight issues. The system shall model citizens, complaints, departments, and service requests as encapsulated classes and manage multiple objects using suitable data types, control structures, arrays, and static members. Different complaint types shall be organized using inheritance and virtual functions to support runtime polymorphism and customized complaint processing. The application shall demonstrate default, parameterized,

and copy constructors, destructors, pointers, and at least two meaningful operator overloads for comparing complaints and generating reports. A menu-driven interface shall provide complaint registration, department assignment, status updates, complaint comparison, resolution tracking, and service-performance report generation.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Requirement Understanding, Data Types, Operators & Control Structures (CO1)	10	Correctly identifies the complaint-management workflow; uses suitable data types, operators, conditional statements and loops efficiently.	Identifies most requirements with minor gaps; uses data types and control structures appropriately	Basic requirements identified; some errors or inefficiencies in data types and control structures	Requirements poorly understood; frequent errors in data types, operators or control flow.
Class Design: Encapsulation, Access Specifiers & Member Functions (CO2)	15	Well-designed classes for proper encapsulation, access specifiers and cohesive member functions.	Classes are appropriately designed with minor issues in encapsulation or member functions.	Classes are present but encapsulation is weak or member functions are partially implemented.	Poor class design; inappropriate access specifiers and inadequate separation of data and behavior.

Static Members & Arrays of Objects (CO2)	10	Correctly uses static members for shared complaint statistics and arrays/collections of objects to manage multiple citizens, complaints and requests.	Static members and arrays of objects are implemented with minor logical errors.	Static members or arrays are partially implemented or underused.	Static members and arrays of objects are missing or non-functional.
Inheritance Hierarchy & Polymorphism (Virtual Functions) (CO4)	15	Clearly implements an inheritance hierarchy for different complaint/service types; virtual functions effectively provide runtime polymorphism for complaint processing and priority handling.	Inheritance and virtual functions are correctly implemented with limited polymorphic behaviour.	Basic inheritance is implemented, but overriding or polymorphism is incomplete.	No meaningful inheritance or polymorphism is implemented.
Constructors, Destructors & Operator Overloading (CO3)	15	Correctly implements default, parameterized and copy constructors; destructor performs appropriate resource cleanup; at least two	Most constructors and at least one operator overload are correctly implemented with minor issues..	Some constructor types or one operator overload is implemented; destructor functionality is incomplete.	Constructors, destructor and operator overloading are missing, incorrect or cause runtime errors..

		meaningful operator overloads are correctly implemented for complaint comparison/report formatting.			
Menu-Driven Functionality, Correctness & Report Generation	25	All features—complaint registration, categorization, prioritization, department assignment, status updates, resolution tracking, comparison and service reports—work correctly through a clear menu-driven interface.	Most features work correctly with minor logical or formatting errors.	Core functions work, but some features such as prioritization, tracking or reporting are incomplete.	Program is unreliable; major features are missing or non-functional.
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 7/11/12 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; sustainability connection and challenges mentioned but lack depth.	Reflection present but generic; limited justification of design or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no real design/SDG/learning connection.

Urban Complaint Management and Service Request System

1. Objective

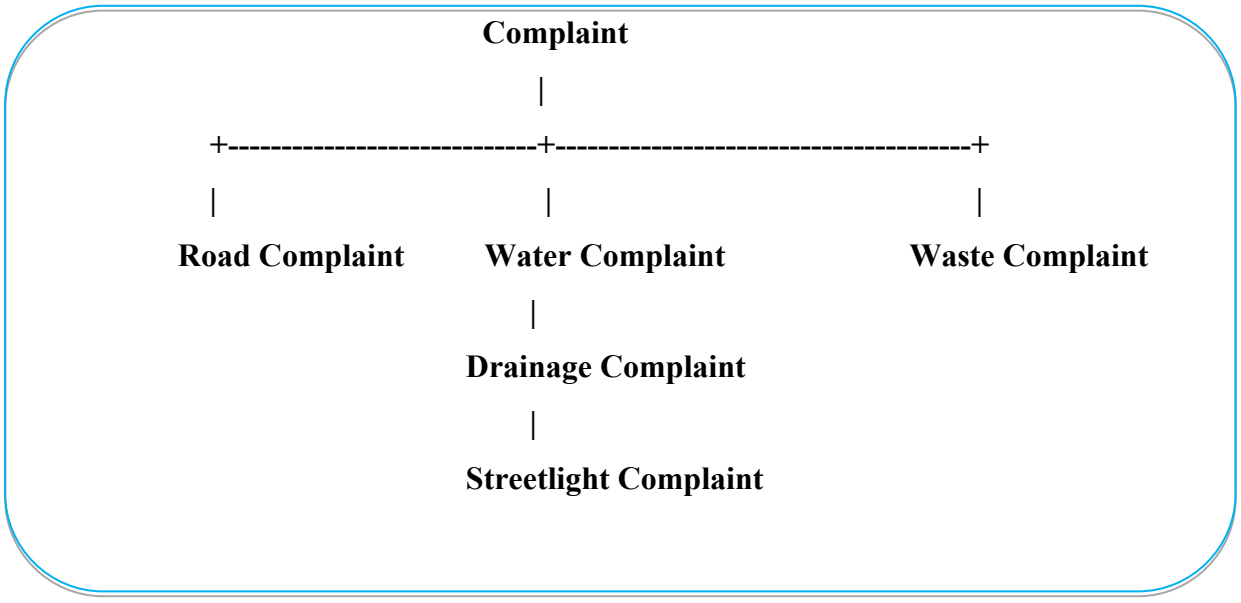
The objective is to develop a menu-driven C++ application that manages civic complaints such as:

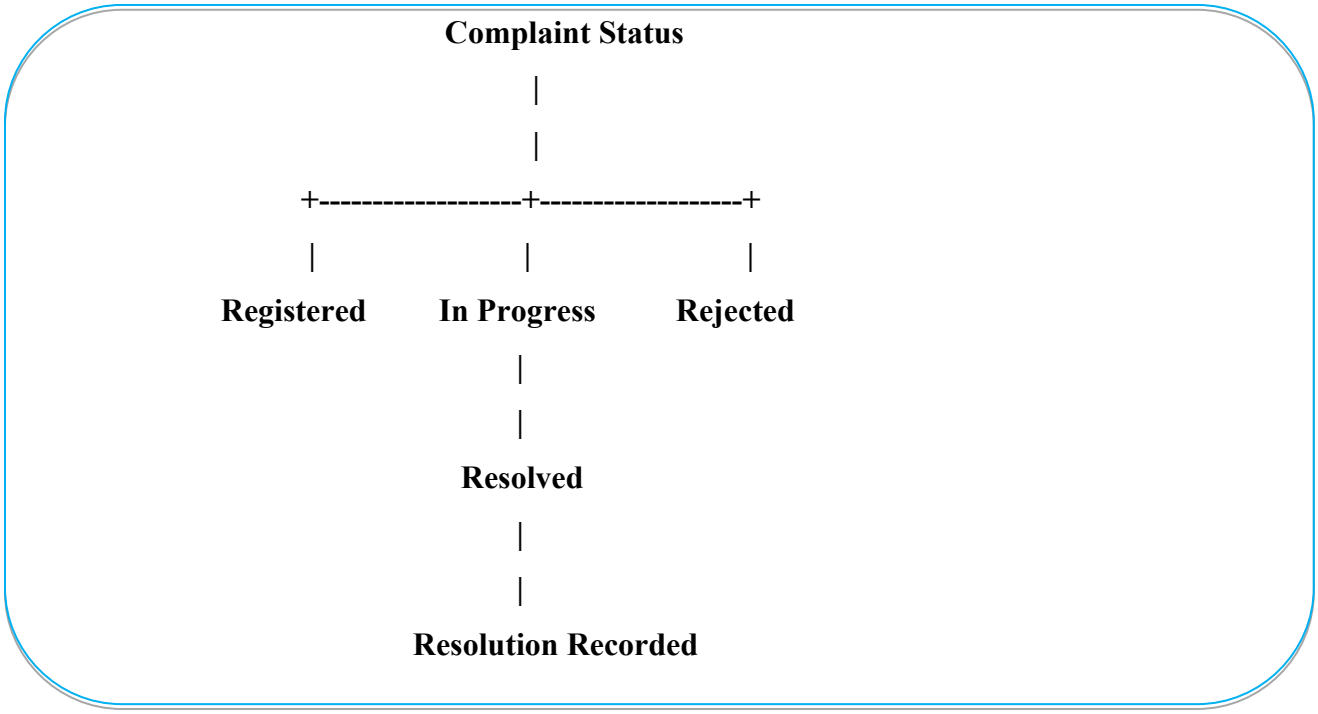
- Road damage
- Water supply
- Waste management
- Drainage
- Streetlight issues

The system allows citizens to register complaints, assign them to departments, prioritize them, update their status, compare complaints, track resolutions, and generate service-performance reports.

2. Proposed Class Design

We will use the following hierarchy:





3. Pseudocode

START

Create arrays for Citizens and Complaint pointers

Initialize complaint counter

REPEAT

Display main menu

1. Register Complaint
2. Display All Complaints
3. Assign Department
4. Update Complaint Status

5. Compare Two Complaints
6. Track Resolution
7. Generate Service Performance Report
8. Display Complaint Statistics
9. Exit

Read user's choice

IF choice = 1
    Read citizen details
    Read complaint type
    Read complaint description
    Create appropriate derived Complaint object
    Store object using pointer
    Calculate priority
    Increment complaint count

ELSE IF choice = 2
    Display all complaints

ELSE IF choice = 3
    Enter complaint ID
    Assign appropriate department

ELSE IF choice = 4
    Enter complaint ID
    Change complaint status

ELSE IF choice = 5
    Enter two complaint IDs
    Compare complaint priority using overloaded operator

ELSE IF choice = 6
Enter complaint ID
Display resolution information

ELSE IF choice = 7
Generate service-performance report

ELSE IF choice = 8
Display total complaint statistics

ELSE IF choice = 9
Exit program

ELSE
Display invalid choice

UNTIL choice = 9

Delete dynamically allocated complaint objects

END

4 PROGRAM:

```
#include <iostream>
#include <string>
#include <iomanip>

using namespace std;
```

```
// ============================================================
// CITIZEN CLASS
// ============================================================

class Citizen
{
private:
    int citizenId;
    string name;
    string phone;

public:

    // Default Constructor
    Citizen()
    {
        citizenId = 0;
        name = "Unknown";
        phone = "Not Available";
    }

    // Parameterized Constructor
    Citizen(int id, string n, string p)
    {
        citizenId = id;
        name = n;
        phone = p;
    }

    void display() const
```

```
    {
        cout << "Citizen ID : " << citizenId << endl;
        cout << "Name       : " << name << endl;
        cout << "Phone      : " << phone << endl;
    }

    int getId() const
    {
        return citizenId;
    }

    string getName() const
    {
        return name;
    }
};



// ============================================================
// ABSTRACT BASE CLASS - COMPLAINT
// ============================================================

class Complaint
{
protected:
    int complaintId;
    Citizen citizen;
    string description;
    string department;
    string status;
    string resolution;
```

```
    int priority;

public:

    // Static members
    static int totalComplaints;
    static int resolvedComplaints;

    // Default Constructor
    Complaint()
    {
        complaintId = 0;
        description = "";
        department = "Not Assigned";
        status = "Registered";
        resolution = "Pending";
        priority = 1;

        totalComplaints++;
    }

    // Parameterized Constructor
    Complaint(int id, Citizen c, string desc)
    {
        complaintId = id;
        citizen = c;
        description = desc;
        department = "Not Assigned";
        status = "Registered";
        resolution = "Pending";
        priority = 1;
```

```
    totalComplaints++;
}

// Copy Constructor
Complaint(const Complaint &obj)
{
    complaintId = obj.complaintId;
    citizen = obj.citizen;
    description = obj.description;
    department = obj.department;
    status = obj.status;
    resolution = obj.resolution;
    priority = obj.priority;
}

// Virtual Destructor
virtual ~Complaint()
{
    // No dynamic memory to release
}

// Pure virtual functions
virtual void processComplaint() = 0;
virtual string getType() const = 0;

// Virtual display function
virtual void display() const
{
    cout << "\n---------------------------------------" << endl;
    cout << "Complaint ID : " << complaintId << endl;
```

```
    cout << "Type        : " << getType() << endl;
    cout << "Description  : " << description << endl;
    cout << "Department   : " << department << endl;
    cout << "Priority     : " << priority << endl;
    cout << "Status       : " << status << endl;
    cout << "Resolution   : " << resolution << endl;
    cout << "-------------------------------------" << endl;
}

// Operator overload 1: compare priority
bool operator>(const Complaint &obj) const
{
    return priority > obj.priority;
}

// Operator overload 2: compare complaint IDs
bool operator==(const Complaint &obj) const
{
    return complaintId == obj.complaintId;
}

// Assign department
void assignDepartment(string dept)
{
    department = dept;
}

// Update status
void updateStatus(string newStatus)
{
    status = newStatus;
```

```
    if (status == "Resolved")
    {
        resolution = "Complaint resolved successfully";
        resolvedComplaints++;
    }
}

// Resolution tracking
void setResolution(string r)
{
    resolution = r;
}

int getId() const
{
    return complaintId;
}

int getPriority() const
{
    return priority;
}

string getStatus() const
{
    return status;
}

string getDepartment() const
{
```

```
        return department;
    }

    static void displayStatistics()
    {
        cout << "\n========== COMPLAINT STATISTICS ==========" << endl;
        cout << "Total Complaints   : " << totalComplaints << endl;
        cout << "Resolved Complaints: " << resolvedComplaints << endl;

        if (totalComplaints > 0)
        {
            double percentage =
                ((double)resolvedComplaints / totalComplaints) * 100;

            cout << fixed << setprecision(2);
            cout << "Resolution Rate    : "
                 << percentage << "%" << endl;
        }

        cout << "==========================================" << endl;
    }
};


// Initialize static members
int Complaint::totalComplaints = 0;
int Complaint::resolvedComplaints = 0;


// ============================================================
// ROAD COMPLAINT
```

```
// ============================================================

class RoadComplaint : public Complaint
{
public:

   RoadComplaint(int id, Citizen c, string desc)
      : Complaint(id, c, desc)
   {
      priority = 4;
   }

   void processComplaint() override
   {
      department = "Road and Transport Department";

      if (description.find("accident") != string::npos)
         priority = 5;
      else
         priority = 4;
   }

   string getType() const override
   {
      return "Road Damage";
   }
};



// ============================================================
// WATER COMPLAINT
```

```
// ============================================================

class WaterComplaint : public Complaint
{
public:

    WaterComplaint(int id, Citizen c, string desc)
        : Complaint(id, c, desc)
    {
        priority = 4;
    }

    void processComplaint() override
    {
        department = "Water Supply Department";

        if (description.find("no water") != string::npos)
            priority = 5;
        else
            priority = 4;
    }

    string getType() const override
    {
        return "Water Supply";
    }
};


// ============================================================
// WASTE COMPLAINT
```

```
// ============================================================

class WasteComplaint : public Complaint
{
public:

   WasteComplaint(int id, Citizen c, string desc)
      : Complaint(id, c, desc)
   {
      priority = 3;
   }

   void processComplaint() override
   {
      department = "Waste Management Department";
      priority = 3;
   }

   string getType() const override
   {
      return "Waste Management";
   }
};



// ============================================================
// DRAINAGE COMPLAINT
// ============================================================

class DrainageComplaint : public Complaint
{
```

```
public:

    DrainageComplaint(int id, Citizen c, string desc)
        : Complaint(id, c, desc)
    {
        priority = 5;
    }

    void processComplaint() override
    {
        department = "Drainage Department";
        priority = 5;
    }

    string getType() const override
    {
        return "Drainage";
    }
};



// ================================================================
// STREETLIGHT COMPLAINT
// ================================================================

class StreetlightComplaint : public Complaint
{
public:

    StreetlightComplaint(int id, Citizen c, string desc)
        : Complaint(id, c, desc)
```

```
    {
        priority = 2;
    }

    void processComplaint() override
    {
        department = "Electrical Department";
        priority = 2;
    }

    string getType() const override
    {
        return "Streetlight";
    }
};



// ================================================================
// GLOBAL CONSTANT
// ================================================================

const int MAX_COMPLAINTS = 100;



// ================================================================
// FIND COMPLAINT
// ================================================================

Complaint* findComplaint(Complaint* complaints[], int count, int id)
{
    for (int i = 0; i < count; i++)
```

```
    {
        if (complaints[i]->getId() == id)
            return complaints[i];
    }

    return NULL;
}


// ============================================================
// REGISTER COMPLAINT
// ============================================================

void registerComplaint(
    Complaint* complaints[],
    int &count,
    Citizen citizens[],
    int &citizenCount)
{
    if (count >= MAX_COMPLAINTS)
    {
        cout << "\nComplaint storage is full!" << endl;
        return;
    }

    int citizenId;
    string name;
    string phone;

    cout << "\n========== REGISTER COMPLAINT ==========" << endl;
```

```
cout << "Enter Citizen ID: ";
cin >> citizenId;

cin.ignore();

cout << "Enter Citizen Name: ";
getline(cin, name);

cout << "Enter Phone Number: ";
getline(cin, phone);

citizens[citizenCount] =
   Citizen(citizenId, name, phone);

citizenCount++;

int type;

cout << "\nSelect Complaint Type:" << endl;
cout << "1. Road Damage" << endl;
cout << "2. Water Supply" << endl;
cout << "3. Waste Management" << endl;
cout << "4. Drainage" << endl;
cout << "5. Streetlight" << endl;

cout << "Enter choice: ";
cin >> type;

cin.ignore();

string description;
```

```
cout << "Enter Complaint Description: ";
getline(cin, description);

int complaintId = count + 1;

Complaint* newComplaint = NULL;

// Runtime polymorphism
switch (type)
{
   case 1:
      newComplaint =
         new RoadComplaint(
            complaintId,
            citizens[citizenCount - 1],
            description);
      break;

   case 2:
      newComplaint =
         new WaterComplaint(
            complaintId,
            citizens[citizenCount - 1],
            description);
      break;

   case 3:
      newComplaint =
         new WasteComplaint(
            complaintId,
```

```
            citizens[citizenCount - 1],
            description);
        break;

    case 4:
        newComplaint =
            new DrainageComplaint(
            complaintId,
            citizens[citizenCount - 1],
            description);
        break;

    case 5:
        newComplaint =
            new StreetlightComplaint(
            complaintId,
            citizens[citizenCount - 1],
            description);
        break;

    default:
        cout << "Invalid complaint type!" << endl;
        return;
}

newComplaint->processComplaint();

complaints[count] = newComplaint;
count++;

cout << "\nComplaint registered successfully!" << endl;
```

```
    cout << "Complaint ID: " << complaintId << endl;
}


// ============================================================
// DISPLAY ALL COMPLAINTS
// ============================================================

void displayAllComplaints(
    Complaint* complaints[],
    int count)
{
    if (count == 0)
    {
        cout << "\nNo complaints registered." << endl;
        return;
    }

    cout << "\n========== ALL COMPLAINTS ==========" << endl;

    for (int i = 0; i < count; i++)
    {
        complaints[i]->display();
    }
}


// ============================================================
// ASSIGN DEPARTMENT
// ============================================================
```

```
void assignDepartment(
    Complaint* complaints[],
    int count)
{
    int id;

    cout << "\nEnter Complaint ID: ";
    cin >> id;

    Complaint* complaint =
        findComplaint(complaints, count, id);

    if (complaint == NULL)
    {
        cout << "Complaint not found!" << endl;
        return;
    }

    string department;

    cin.ignore();

    cout << "Enter Department: ";
    getline(cin, department);

    complaint->assignDepartment(department);

    cout << "Department assigned successfully." << endl;
}
```

```
// ============================================================
// UPDATE STATUS
// ============================================================

void updateComplaintStatus(
    Complaint* complaints[],
    int count)
{
    int id;

    cout << "\nEnter Complaint ID: ";
    cin >> id;

    Complaint* complaint =
        findComplaint(complaints, count, id);

    if (complaint == NULL)
    {
        cout << "Complaint not found!" << endl;
        return;
    }

    int choice;

    cout << "\nSelect New Status:" << endl;
    cout << "1. Registered" << endl;
    cout << "2. In Progress" << endl;
    cout << "3. Resolved" << endl;
    cout << "4. Rejected" << endl;

    cout << "Enter choice: ";
```

```
    cin >> choice;

    string status;

    switch (choice)
    {
        case 1:
            status = "Registered";
            break;

        case 2:
            status = "In Progress";
            break;

        case 3:
            status = "Resolved";
            break;

        case 4:
            status = "Rejected";
            break;

        default:
            cout << "Invalid status!" << endl;
            return;
    }

    complaint->updateStatus(status);

    cout << "Status updated successfully." << endl;
}
```

```
// ============================================================
// COMPARE COMPLAINTS
// ============================================================

void compareComplaints(
    Complaint* complaints[],
    int count)
{
    int id1, id2;

    cout << "\nEnter first Complaint ID: ";
    cin >> id1;

    cout << "Enter second Complaint ID: ";
    cin >> id2;

    Complaint* c1 =
        findComplaint(complaints, count, id1);

    Complaint* c2 =
        findComplaint(complaints, count, id2);

    if (c1 == NULL || c2 == NULL)
    {
        cout << "One or both complaints not found!" << endl;
        return;
    }

    cout << "\n========== COMPARISON ==========" << endl;
```

```
    if (*c1 > *c2)
    {
        cout << "Complaint " << id1
             << " has higher priority." << endl;
    }
    else if (*c2 > *c1)
    {
        cout << "Complaint " << id2
             << " has higher priority." << endl;
    }
    else
    {
        cout << "Both complaints have equal priority."
             << endl;
    }

    if (*c1 == *c2)
    {
        cout << "Both objects have the same Complaint ID."
             << endl;
    }
    else
    {
        cout << "Complaint IDs are different." << endl;
    }
}


// ========================================================
// RESOLUTION TRACKING
```

```
// ============================================================

void trackResolution(
    Complaint* complaints[],
    int count)
{
    int id;

    cout << "\nEnter Complaint ID: ";
    cin >> id;

    Complaint* complaint =
        findComplaint(complaints, count, id);

    if (complaint == NULL)
    {
        cout << "Complaint not found!" << endl;
        return;
    }

    cout << "\n========== RESOLUTION TRACKING ==========="
         << endl;

    cout << "Complaint ID : "
         << complaint->getId() << endl;

    cout << "Department   : "
         << complaint->getDepartment() << endl;

    cout << "Status       : "
         << complaint->getStatus() << endl;
```

```
}


// ============================================================
// SERVICE PERFORMANCE REPORT
// ============================================================

void servicePerformanceReport(
    Complaint* complaints[],
    int count)
{
    int registered = 0;
    int inProgress = 0;
    int resolved = 0;
    int rejected = 0;

    cout << "\n========== SERVICE PERFORMANCE REPORT ==========="
         << endl;

    for (int i = 0; i < count; i++)
    {
        string status = complaints[i]->getStatus();

        if (status == "Registered")
            registered++;

        else if (status == "In Progress")
            inProgress++;

        else if (status == "Resolved")
            resolved++;
```

```
        else if (status == "Rejected")
            rejected++;
    }

    cout << "Total Complaints : " << count << endl;
    cout << "Registered       : " << registered << endl;
    cout << "In Progress      : " << inProgress << endl;
    cout << "Resolved         : " << resolved << endl;
    cout << "Rejected         : " << rejected << endl;

    if (count > 0)
    {
        double resolutionRate =
            ((double)resolved / count) * 100;

        cout << fixed << setprecision(2);

        cout << "Resolution Rate  : "
             << resolutionRate << "%" << endl;
    }

    cout << "================================================="
         << endl;
}


// ============================================================
// MAIN FUNCTION
// ============================================================
```

```
int main()
{
   Complaint* complaints[MAX_COMPLAINTS];

   Citizen citizens[MAX_COMPLAINTS];

   int complaintCount = 0;
   int citizenCount = 0;

   int choice;

   cout << "================================================="
      << endl;

   cout << "   URBAN COMPLAINT MANAGEMENT SYSTEM"
      << endl;

   cout << "================================================="
      << endl;

   do
   {
      cout << "\n\n========== MAIN MENU ==========" << endl;

      cout << "1. Register Complaint" << endl;
      cout << "2. Display All Complaints" << endl;
      cout << "3. Assign Department" << endl;
      cout << "4. Update Complaint Status" << endl;
      cout << "5. Compare Two Complaints" << endl;
      cout << "6. Track Resolution" << endl;
      cout << "7. Service Performance Report" << endl;
```

```
cout << "8. Complaint Statistics" << endl;
cout << "9. Exit" << endl;

cout << "Enter your choice: ";
cin >> choice;

switch (choice)
{
   case 1:
      registerComplaint(
         complaints,
         complaintCount,
         citizens,
         citizenCount);
      break;

   case 2:
      displayAllComplaints(
         complaints,
         complaintCount);
      break;

   case 3:
      assignDepartment(
         complaints,
         complaintCount);
      break;

   case 4:
      updateComplaintStatus(
         complaints,
```

```
            complaintCount);
        break;

    case 5:
        compareComplaints(
            complaints,
            complaintCount);
        break;

    case 6:
        trackResolution(
            complaints,
            complaintCount);
        break;

    case 7:
        servicePerformanceReport(
            complaints,
            complaintCount);
        break;

    case 8:
        Complaint::displayStatistics();
        break;

    case 9:
        cout << "\nExiting system..." << endl;
        break;

    default:
        cout << "\nInvalid choice!"
```

```
                << endl;
        }

    } while (choice != 9);



    // Free dynamically allocated memory
    for (int i = 0; i < complaintCount; i++)
    {
        delete complaints[i];
    }

    cout << "\nAll complaint objects destroyed."
         << endl;

    return 0;
}
```

5)Implementation screen shots:

```
==============================================
   URBAN COMPLAINT MANAGEMENT SYSTEM
==============================================


========== MAIN MENU ==========
1. Register Complaint
2. Display All Complaints
3. Assign Department
4. Update Complaint Status
5. Compare Two Complaints
6. Track Resolution
7. Service Performance Report
8. Complaint Statistics
9. Exit
Enter your choice: 1
```

```
========== REGISTER COMPLAINT ==========
Enter Citizen ID: 24
Enter Citizen Name: suvan
Enter Phone Number: 9089

Select Complaint Type:
1. Road Damage
2. Water Supply
3. Waste Management
4. Drainage
5. Streetlight
Enter choice: 1
Enter Complaint Description: pathole

Complaint registered successfully!
Complaint ID: 1
```

ID	Type	Department	Priority	Status
1	Road Damage	Road and Transport Dept	4-High	Registered
2	Water Supply	Water Supply Dept	4-High	Registered
3	Drainage	Drainage Dept	**5-Critical**	Registered
4	Streetlight	Electrical Dept	2-Medium	Registered
5	Waste Management	Waste Management Dept	3-Moderate	Registered

ID	Complaint ID	Type	Department	Priority	Status	Date Registered	Last Updated
1	C001	Road Damage	Road and Transport Dept	4-High	Completed	01-05-2025	07-05-2025
2	C002	Water Supply	Water Supply Dept	4-High	Completed	02-05-2025	08-05-2025
3	C003	Drainage	Drainage Dept	5-Critical	Ongoing	03-05-2025	09-05-2025
4	C004	Streetlight	Electrical Dept	2-Medium	Ongoing	04-05-2025	09-05-2025
5	C005	Waste Management	Waste Management Dept	3-Moderate	Completed	05-05-2025	10-05-2025
6	C006	Water Supply	Water Supply Dept	4-High	Ongoing	06-05-2025	10-05-2025
7	C007	Road Damage	Road and Transport Dept	3-Moderate	Completed	07-05-2025	11-05-2025
8	C008	Drainage	Drainage Dept	5-Critical	Ongoing	08-05-2025	11-05-2025

6)Hardware and Software Components:

Component	Specification / Purpose
Computer / Laptop	Used for developing, compiling, testing, and executing the application
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB; 8 GB recommended
Storage	Minimum 500 MB free space for source code, development tools, and database
Keyboard & Mouse	Used for entering citizen and complaint information

Component	Specification / Purpose
Display Monitor	Used to interact with the console-based application

7). Programming Language used:

Category	Technology Used
Programming Language	**C++**
Database	**SQLite**
Development Environment	**VS Code**
Version Control	**GitHub**

8). Individual Contribution:

My individual technical contribution to the Urban Complaint Management and Service Request System focused on the design and implementation of the complete C++ application. I developed the object-oriented class architecture using the Citizen, abstract Complaint, derived complaint classes, and Database Manager classes. I implemented default, parameterized, and copy constructors, virtual destructors, inheritance, pure virtual functions, runtime polymorphism using base-class pointers, static members, and operator overloading (> and ==). I also integrated SQLite3 for persistent storage and implemented SQL-based CRUD operations for citizens and complaints, including registration, retrieval, searching, department assignment, status updates, and resolution tracking. Additionally, I implemented priority handling, service-performance reports using SQL aggregation, input validation, exception handling, memory management, and system testing to ensure the application functions correctly.

9) Conclusion:

The Urban Complaint Management and Service Request System successfully demonstrates the application of C++ object-oriented programming concepts to a real-world urban service management problem. The system provides an organized approach for registering, categorizing, prioritizing, assigning,

tracking, and resolving civic complaints. The implementation uses encapsulation, constructors, destructors, inheritance, abstraction, virtual functions, runtime polymorphism, pointers, static members, and operator overloading, along with SQLite database integration for persistent data storage. The menu-driven interface makes the system easy to operate, while database-based reporting supports effective monitoring of complaint resolution and service performance. Overall, the project provides a scalable foundation for improving the efficiency, transparency, and management of urban civic services.